Now i want to automate the regression tests like in uploaded document with qwen3-vl-2b-instruct and qwen2.5-0.5b models which i already downloaded.How can i do that?Provide me simple tests so that i can understand how it will be helpful if i do the regression tests with both the above llm and multimodel.I have following project structure already

ai-regression-tests/ ├─ tests/ │ ├─ test_web_regression.py │ └─ test_ai_visual_regression.py ├─ screenshots/ ├─ reports/ ├─ ai/ [5] [6] [3] │ ├─ foundry_client.py │ └─ prompts.py

Your uploaded document contains two basic Playwright regression tests: a homepage title/navigation check and a Todo MVC add-item check. You can extend that pattern by keeping **deterministic Playwright assertions** for functionality and adding **LLM/VLM checks** for text quality and visual/UI regression.[1]

Recommended approach

Test type	Model	What it checks	Example
Functional regression	None	Exact, reliable UI behavior	Todo count becomes `1` [1]
Text regression	`qwen2.5-0.5b`	Summary, extracted text, expected wording	Page summary mentions “installation”
Visual regression	`qwen3-vl-2b-instruct`	Screenshot contents, layout, visible text, obvious defects	Screenshot shows a heading and no error message

Use the small text model for cheap text checks and the vision model only for screenshots. Keep AI assertions broad—for example, “mentions installation”—rather than requiring exact model wording.

Project layout

```
ai-regression-tests/
├─ tests/
│  ├─ test_web_regression.py
│  └─ test_ai_visual_regression.py
├─ screenshots/
├─ reports/
├─ ai/
```

```
|  ├─ foundry_client.py
|  └─ prompts.py
├─ pytest.ini
└─ requirements.txt
```

Install dependencies

```
pip install pytest playwright requests openai
playwright install chromium
```

`ai/foundry_client.py`

```
import base64
import mimetypes
from pathlib import Path

import requests
from openai import OpenAI

BASE_URL = "http://127.0.0.1:58186/v1"
TEXT_MODEL = "qwen2.5-0.5b"
VISION_MODEL = "qwen3-vl-2b-instruct"

client = OpenAI(
    base_url=BASE_URL,
    api_key="not-needed",
)


def ask_text_model(prompt: str, context: str = "") -> str:
    response = client.chat.completions.create(
        model=TEXT_MODEL,
        messages=[
            {
                "role": "user",
                "content": f"{context}\n\n{prompt}".strip(),
            }
        ],
        temperature=0,
        max_tokens=256,
    )
    return response.choices[^0].message.content.strip()


def ask_vision_model(prompt: str, image_path: Path) -> str:
    media_type, _ = mimetypes.guess_type(image_path)
    media_type = media_type or "image/png"

    image_b64 = base64.b64encode(image_path.read_bytes()).decode("utf-8")

    vision_input = [
        {
            "type": "message",
```

```
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": prompt,
                },
                {
                    "type": "input_image",
                    "image_data": image_b64,
                    "media_type": media_type,
                },
            ],
        }
    ]

    response = client.responses.create(
        model=VISION_MODEL,
        input="placeholder",
        extra_body={
            "input": vision_input,
            "max_output_tokens": 512,
        },
        stream=False,
    )

    return response.output_text.strip()
```

ai/prompts.py

```
PAGE_SUMMARY_PROMPT = """
Summarize the following web page text in 2-3 sentences.
Focus on the main purpose of the page.

PAGE TEXT:
{page_text}
"""

PAGE_CONTENT_CHECK_PROMPT = """
Answer only YES or NO.

Based on the following page text, does the page appear to be about Playwright
and include information about getting started or installation?

PAGE TEXT:
{page_text}
"""

SCREENSHOT_CHECK_PROMPT = """
Look at this web page screenshot.

Answer in this exact format:
VISIBLE_HEADING: <main heading or NONE>
HAS_ERROR: YES or NO
BRIEF_DESCRIPTION: <one sentence>
"""
```

```
TODO_SCREENSHOT_PROMPT = """
Look at this TodoMVC screenshot.

Answer only YES or NO.

Does the screenshot show a todo item with the text '{expected_text}'?
"""
```

tests/test_web_regression.py

These are your deterministic regression tests, based on the examples in your document. [1]

```
import re

from playwright.sync_api import Page, expect

BASE_URL = "https://playwright.dev"


def test_home_page_title(page: Page):
    page.goto(BASE_URL)
    expect(page).to_have_title(re.compile("Playwright"))


def test_get_started_navigation(page: Page):
    page.goto(BASE_URL)
    page.get_by_role("link", name="Get started").click()
    expect(page.get_by_role("heading", name="Installation")).to_be_visible()


def test_add_todo_and_verify_count(page: Page):
    page.goto("https://demo.playwright.dev/todomvc")

    todo_box = page.get_by_placeholder("What needs to be done?")
    todo_box.fill("Learn regression testing")
    todo_box.press("Enter")

    expect(page.get_by_text("Learn regression testing")).to_be_visible()
    expect(page.get_by_test_id("todo-count")).to_contain_text("1")
```

tests/test_ai_visual_regression.py

```
import re
from pathlib import Path

import pytest
from playwright.sync_api import Page

from ai.foundry_client import ask_text_model, ask_vision_model
from ai.prompts import (
    PAGE_CONTENT_CHECK_PROMPT,
    PAGE_SUMMARY_PROMPT,
```

```
    SCREENSHOT_CHECK_PROMPT,
    TODO_SCREENSHOT_PROMPT,
)

SCREENSHOTS = Path("screenshots")
REPORTS = Path("reports")

SCREENSHOTS.mkdir(exist_ok=True)
REPORTS.mkdir(exist_ok=True)


def save_report(name: str, content: str):
    report_path = REPORTS / f"{name}.txt"
    report_path.write_text(content, encoding="utf-8")
    return report_path


def test_playwright_homepage_text_regression(page: Page):
    page.goto("https://playwright.dev")

    page_text = page.inner_text("body")

    summary = ask_text_model(
        PAGE_SUMMARY_PROMPT.format(page_text=page_text[:4000])
    )

    content_answer = ask_text_model(
        PAGE_CONTENT_CHECK_PROMPT.format(page_text=page_text[:4000])
    )

    save_report(
        "playwright_homepage_text_regression",
        f"Summary:\n{summary}\n\nContent check:\n{content_answer}",
    )

    assert "YES" in content_answer.upper(), (
        f"AI did not recognize expected Playwright content.\n\n{content_answer}"
    )


def test_playwright_homepage_visual_regression(page: Page):
    page.goto("https://playwright.dev")

    screenshot_path = SCREENSHOTS / "playwright_home.png"
    page.screenshot(path=screenshot_path, full_page=True)

    ai_result = ask_vision_model(
        SCREENSHOT_CHECK_PROMPT,
        screenshot_path,
    )

    save_report(
        "playwright_homepage_visual_regression",
        ai_result,
    )
```

```
    assert "VISIBLE_HEADING:" in ai_result
    assert "HAS_ERROR: NO" in ai_result.upper()


def test_todo_item_visible_in_screenshot(page: Page):
    expected_text = "Learn regression testing"

    page.goto("https://demo.playwright.dev/todomvc")

    todo_box = page.get_by_placeholder("What needs to be done?")
    todo_box.fill(expected_text)
    todo_box.press("Enter")

    screenshot_path = SCREENSHOTS / "todo_item.png"
    page.screenshot(path=screenshot_path)

    ai_result = ask_vision_model(
        TODO_SCREENSHOT_PROMPT.format(expected_text=expected_text),
        screenshot_path,
    )

    save_report(
        "todo_item_visual_regression",
        ai_result,
    )

    assert "YES" in ai_result.upper(), (
        f"Vision model could not confirm the todo item.\n\n{ai_result}"
    )
```

`pytest.ini`

```
[pytest]
testpaths = tests
addopts = -v -s
```

Run the tests

From the `ai-regression-tests/` folder:

```
pytest
```

Run only the AI tests:

```
pytest tests/test_ai_visual_regression.py
```

Run only the standard web regression tests:

```
pytest tests/test_web_regression.py
```

What each test demonstrates

- `test_playwright_homepage_text_regression`: sends page text to `qwen2.5-0.5b`, asks for a summary, and checks whether the page is about Playwright/installation.
- `test_playwright_homepage_visual_regression`: takes a full-page screenshot and asks `qwen3-vl-2b-instruct` to identify the heading and detect an error state.
- `test_todo_item_visible_in_screenshot`: performs the functional Todo action, screenshots the result, and asks the vision model whether the expected item is visible.

This gives you a practical regression setup: Playwright proves the workflow works, while the text LLM checks semantic content and the VLM checks what is actually rendered on screen. Your existing document's tests remain the reliable baseline; the AI checks add a second layer for content and visual regressions. [1]

⁂

1. simple-tests.md